



哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

規格嚴格 功夫到家
1920 — 2017

Chapter 8: Stream Input and Output



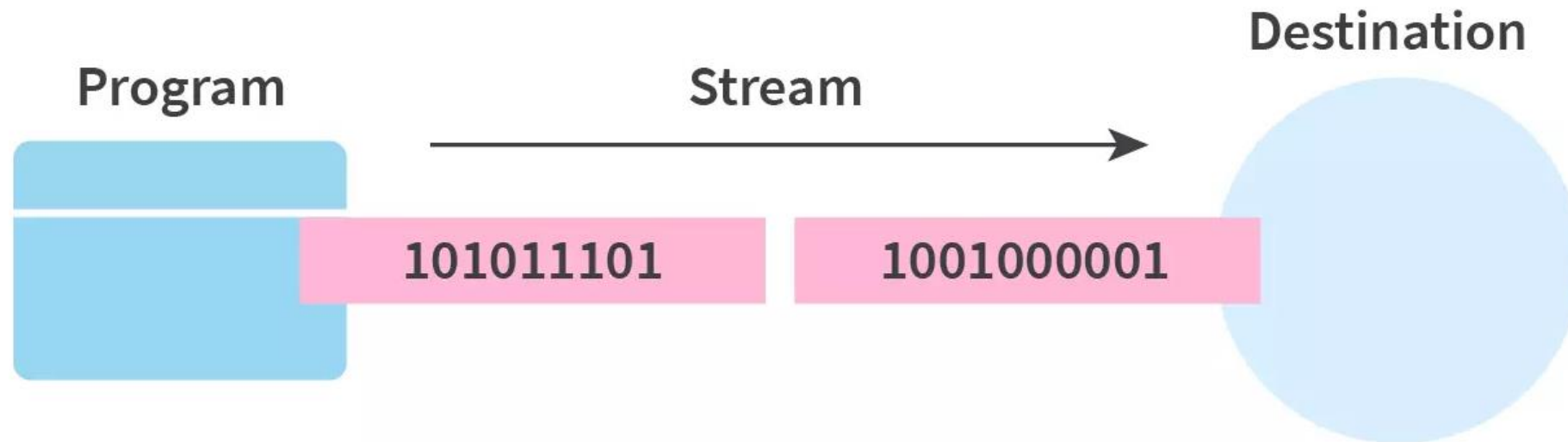
Chapter 8: Stream Input and Output

- Stream
- Input and output streams
- Java stream inheritance framework
- File operation
- Object input/output streams and serialization
- Data access object pattern



Stream

- ❑ Streaming is an abstract concept: a stream is an ordered set of data sequences that carry data from one place to another
- ❑ Abstraction of input and output devices
 - In Java programs, the input/output operations of data are carried out in the form of "streams".
 - Devices can be files, networks, memory, etc.

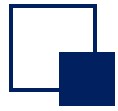




Stream

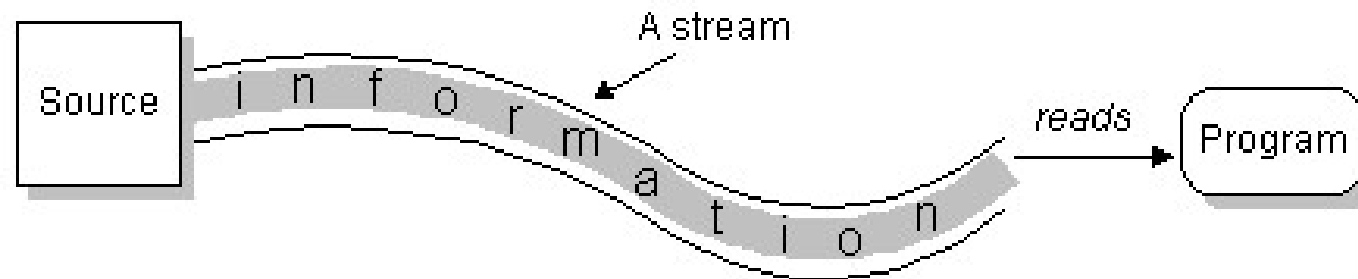
□ Classification of streams

- According to the direction of the flow, it is mainly divided into two categories: input streams and output streams.
- Data streams are divided into byte streams and character streams according to different data units.
 - Byte stream: The basic unit is bytes
 - Character flow: The basic unit is characters
- According to function, it can be divided into node flow and processing flow.
 - Node flow: Data can be read and written from or to a specific place (node), directly connected to the data source
 - Processing stream: A node flow class is called through an indirect flow class (used to wrap the node flow) to read and write various types of data more flexibly and conveniently



Stream

- ❑ Flow is directional, as for whether it is an input stream or an output stream is a relative concept, generally based on the program:
 - If the flow of data is from the program to the device, we call it an output stream,
 - Instead, we call it an input stream.
- ❑ The flow can be imagined as a "water flow pipe", and the water flow is formed in this pipe, and the concept of direction naturally appears.





Chapter 8: Stream Input and Output

- Stream
- Input and output streams
- Java stream inheritance framework
- File operation
- Object input/output streams and serialization
- Data access object pattern

Java system flow

- ❑ Every Java program comes with a system flow, and the system flow corresponds to the class `java.lang.System`
- ❑ Contains three subclasses:
 - `System.in`: Standard input flow, the default device is the keyboard.
 - `System.out`: Standard output stream, the default device is the console.
 - `System.err`: Standard error flow, default device is console.

Read console input

- ❑ Java's console input is done by `System.in`.
- ❑ To get a character stream bound to the console, you can create a character stream by wrapping `System.in` in a `BufferedReader` object.
- ❑ Here's the basic syntax for creating a `BufferedReader`:

```
BufferedReader br = new BufferedReader(new  
    InputStreamReader(System.in));
```

- ❑ Once the `BufferedReader` object is created, we can use the `read()` method to read a character from the console, or the `readLine()` method to read a string.



Read multi-character inputs from the console

- ❑ The following program demonstrates the `read()` method to continuously read characters from the console until the user enters `q`.

```
//Use BufferedReader to read characters in the console
import java.io.*;
public class BRRead {
    public static void main(String[] args) throws IOException {
        char c;
        // Create a BufferedReader using System.in
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        System.out.println("Enter a character and press the 'q' key to exit.");
        // Read characters
        do {
            c = (char) br.read();
            System.out.println(c);
        } while (c != 'q');
    }
}
```

System.out contains the methods:

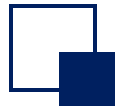
- `print()`: No line breaks
- `println()`: Wrap the line
- `write()`: rarely used



Read multi-character inputs from the console

□ The results of the above example are as follows:

```
1.Enter a character and press the 'q' key to exit.  
2.runoob  
3.r  
4.u  
5.n  
6.o  
7.o  
8.b  
9.  
10.  
11.q
```



Read the string from the console

- ❑ The following program reads and displays lines of characters with the `readline()` method until the word "end" is entered.

```
//Use BufferedReader to read characters in the console
import java.io.*;
public class BRReadLines {
    public static void main(String[] args) throws IOException {
        // Create a BufferedReader using System.in
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        String str;
        System.out.println("Enter lines of text.");
        System.out.println("Enter 'end' to quit.");
        do {
            str = br.readLine();
            System.out.println(str);
        } while (!str.equals("end"));
    }
}
```

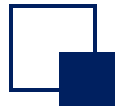
Read the string from the console

❑ The above example compiles and runs the following results:

```
1.Enter lines of text.  
2.Enter 'end' to quit.  
3.This is line one  
4.This is line one  
5.This is line two  
6.This is line two  
7.end
```

Input and output of the file

- ❑ The two important streams that will be discussed below are:
 - **FileInputStream**
 - **FileOutputStream**。



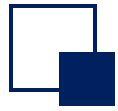
FileInputStream

- ❑ The stream is used to read data from a file, and its objects can be created with the keyword `new`.
- ❑ There are several construction methods that can be used to create objects.
 - You can use the file name of the string type to create an input stream object to read the file:

```
InputStream f = new FileInputStream("C:/java/hello");
```

- You can also use a file object to create an input stream object to read the file. First, we need to use the `File()` method to create a file object:

```
File f = new File("C:/java/hello");  
InputStream in = new FileInputStream(f);
```



FileOutputStream

- ❑ This class is used to create a file and write data to it.
- ❑ If the destination file does not exist before the stream opens the file for output, the stream creates the file.
- ❑ There are two construction methods that can be used to create a `FileOutputStream` object.
 - Use the file name of the string type to create an output stream object:

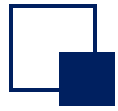
```
OutputStream f = new FileOutputStream("C:/java/hello");
```

- Use a file object to create an output stream to write a file. We first need to create a file object using the `File()` method:

```
File f = new File("C:/java/hello");  
OutputStream fOut = new FileOutputStream(f);
```

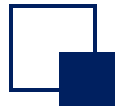
File input and output examples

```
1.import java.io.*;
2.public class FileStreamTest {
3.    public static void main(String[] args) throws IOException {
4.        File f = new File("a.txt");
5.        FileOutputStream fop = new FileOutputStream(f);
6.        // Construct a FileOutputStream object, and if the file does not exist, it
        // will be automatically created
7.
8.        OutputStreamWriter writer = new OutputStreamWriter(fop, "UTF-8");
9.        // Construct an OutputStreamWriter object, and the parameters can be specified
        // to be encoded, which is the default encoding of the operating system
10.
11.        writer.append("中文输入");
12.        // Write to the buffer
13.
14.        writer.append("\r\n");
15.        // Line break
16.
17.        writer.append("English");
18.
```



File input and output instances

```
19.         writer.close();
20.         // Closing the write stream will also write the buffer contents
to the file, so the comment above is removed
21.         fop.close();
22.         // Turn off the output stream to free up system resources
23.
24.         FileInputStream fip = new FileInputStream(f);
25.         // create FileInputStream object
26.
27.         InputStreamReader reader = new InputStreamReader(fip, "UTF-8");
28.         // Build an InputStreamReader object with the same encoding as
writing
29.
30.         StringBuffer sb = new StringBuffer();
31.         while (reader.ready()) {
32.             sb.append((char) reader.read());
33.             // Convert to char and add it to the StringBuffer object
34.         }
```



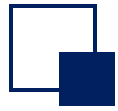
File input and output instances

```
35.         System.out.println(sb.toString());
36.         reader.close();
37.         // Close the read stream
38.
39.         fip.close();
40.         // Turn off the input stream to free up system resources
41.
42.     }
43. }
```



Chapter 8: Stream Input and Output

- Stream
- Input and output streams
- **Java stream inheritance framework**
- File operation
- Object input/output streams and serialization
- Data access object pattern



The complete stream family

- ❑ Unlike C, which only has a single type of FILE*, Java has a stream family with more than 60 input/output stream types!
- ❑ Classification of IO streams
 - Direction: input/reader, output/writer
 - Data:
 - Byte (8bit): Various types, including text, images, sounds, videos, etc
 - Character (mostly 16-bit): Plain text only

What is the content of the Word document that is read?

- ☐ A Character stream
- ☐ B Byte stream
- ☐ C Uncertain

Question

What is the content of the Word document that is read?

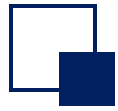
- ☐ A Character stream
- ☒ B Byte stream
- ☐ C Uncertain

submit

The complete stream family

- ❑ All of Java's stream classes are located in `java.io` package, and they are inherited from the following four abstract stream types (four families)

	Byte Stream	Character Stream
Input	InputStream	Reader
Output	OutputStream	Writer



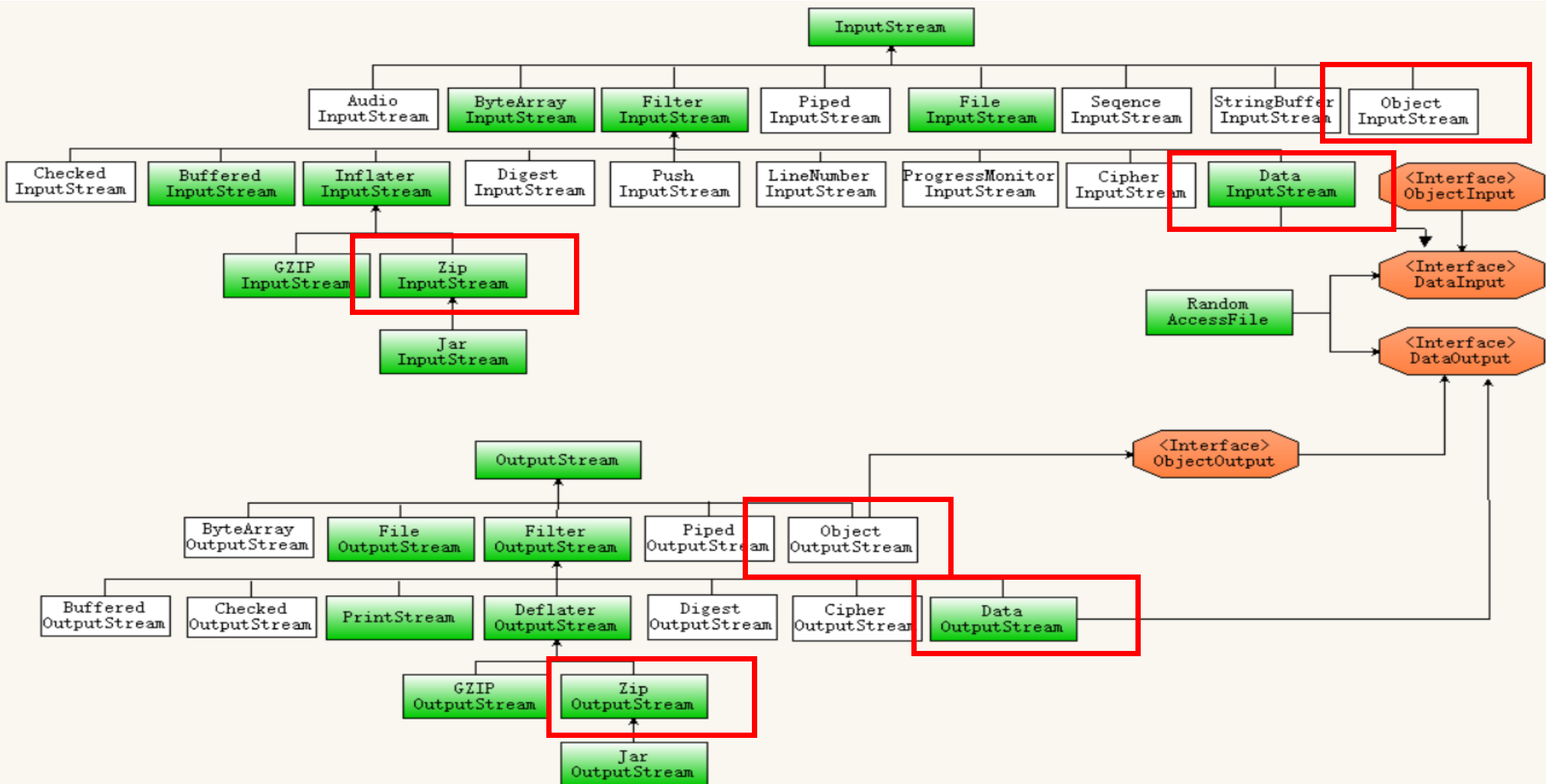
The complete stream family

- ❑ In java, as long as the "class name" ends in Stream, it is a byte stream, and everything that ends in "Reader/Writer" is a character stream
- ❑ All streams implement the java.io.Closeable interface
 - Are all closeable, and all have a close() method
 - After all, a stream is a pipeline that needs to be closed after use, otherwise it will consume a lot of resources
- ❑ Byte and character streams are convertible

```
FileOutputStream fop = new FileOutputStream(f);  
// Construct a FileOutputStream object, and if the file does not exist, it will be automatically created  
OutputStreamWriter writer = new OutputStreamWriter(fop, "UTF-8");  
// Construct an OutputStreamWriter object, and the parameters can be specified to be encoded,  
which is the default encoding of the operating system
```



InputStream and OutputStream hierarchy



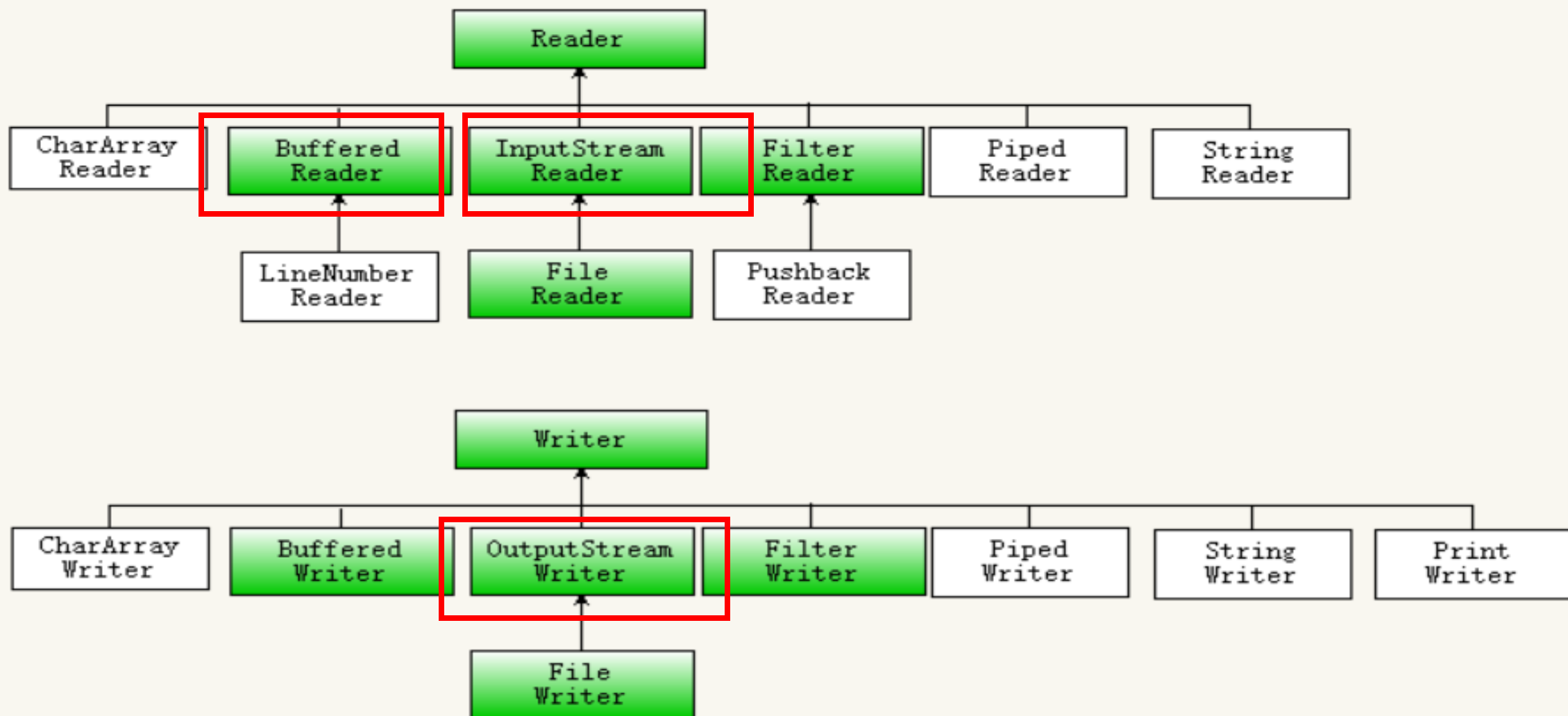


InputStream and OutputStream hierarchy

- ❑ The InputStream and OutputStream classes can read and write individual bytes or arrays of bytes, and these classes form the basis of the hierarchy of input and output streams.
- ❑ To read and write strings and numbers, more powerful subclasses are needed,
 - For example, DataInputStream and DataOutputStream can read and write all basic Java types in binary format.
- ❑ It also contains several useful input/output streams,
 - For example, ZipInputStream and ZipOutputStream can read and write files in our common ZIP compression format.



Reader and Writer hierarchy



Reader and Writer hierarchy

❑ For Unicode text, abstract classes Reader and Writer subclasses can be used

- The basic methods of the Reader/Writer class are similar to those in InputStream/OutputStream.
- Reader/Writer are both character-based

❑ Commonly used subclasses

- BufferedReader: Used to improve efficiency, such as wrapping an InputStreamReader

```
BufferedReader in = new BufferedReader(new InputStreamReader(System.in));
```

- InputStreamReader/OutputStreamReader: Wraps an InputStream to convert a byte-based input stream into a character-based reader for reading characters from a file

```
InputStream inputStream = new FileInputStream("D:\\test\\1.txt");  
Reader inputStreamReader = new InputStreamReader(inputStream);
```



Chapter 8: Stream Input and Output

- Stream
- Input and output streams
- Java stream inheritance framework
- **File operation**
- Object input/output streams and serialization
- Data access object pattern



File operation

- ❑ The previous subsections have already introduced how to read and write data from documents, but file management is much broader than reading and writing.
- ❑ The Path and Files classes encapsulate all the functionality needed to work with the file system on the user's machine.
- ❑ For example, the Files class can be used to remove or rename files, or to query when a file was last modified.
- ❑ This subsection mainly covers the following:
 - Path
 - Read and write files
 - Create files and directories
 - Copy, move, and delete files
 - Get document information



Path

❑ Path represents a sequence of directory names, followed by a file name.

- Paths that start with a root part (e.g. C:\) are absolute paths;
- Otherwise, it is a relative path.

❑ Let's create an absolute path and a relative path respectively:

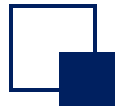
```
Path absolute = Paths.get("c:\data\myfile.txt");  
  
// Paths.get(basePath, relativePath)  
// Assume the current base path "c:\data"  
Path relative = Paths.get("myfile.txt");
```

- The static Paths.get method accepts one or more strings and concatenates them with a path separator for the default file system (UNIX-like file systems are /, Windows is \).



Read and write files

- ❑ Files is a class of tools that manipulate files and contains a large number of methods
- ❑ The Files class can make ordinary file operations faster.
- ❑ The read and write methods provided by Files are limited by memory and can only read and write small files. For large files, you still need a file flow to read and write only a part of the file content at a time.



Read and write documents (read)

❑ All the contents of the file can be easily read in the following way:

- `byte[] bytes = Files.readAllBytes(path);`

❑ We can also read content from a text file in the following way:

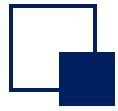
- `var content = Files.readString(path, charset)`, For example:

```
String content = Files.readString(Paths.get("/path/to/file.txt"), StandardCharsets.ISO_8859_1);
```

❑ But if you want to read the file as a sequence of rows, you can call:

- `List<String> lines = Files.readAllLines(path, charset);`

Charset represents a character set, which defaults to UTF-8



Read and write files (write)

❑ If you want to write a string to a file, you can call:

- `Files.writeString(path, content.charset);`

❑ To append content to a specified file, you can call:

- `Files.write(path, content.getBytes(charset), StandardOpenOption.APPEND);`

❑ You can also write a collection of rows to a file with the following statement:

- `Files.write(path, lines, charset);`

```
// Write to binary:
```

```
byte[] data = ...
```

```
Files.write(Paths.get("/path/to/file.txt"), data);
```

```
// Write the text and specify the encoding:
```

```
Files.writeString(Paths.get("/path/to/file.txt"), "Text content...", StandardCharsets.ISO_8859_1);
```

```
// Write text by line:
```

```
List<String> lines = ...
```

```
Files.write(Paths.get("/path/to/file.txt"), lines);
```



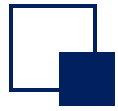
Create files and directories

❑ Creating a new directory can be called:

- `Files.createDirectory(path);`
- All but the last part of the path must already exist.

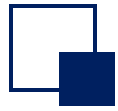
❑ An empty file can be created using the following statement:

- `Files.createFile(path);`
- If the file already exists, then this call throws an exception.



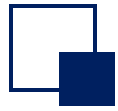
Copy, move, and delete files

- ❑ Copying files from one location to another can be called directly:
 - `Files.copy(fromPath, toPath);`
- ❑ Moving a file (i.e., copying and deleting the original file) can be called:
 - `Files.move(fromPath, toPath);`
- ❑ If the target path already exists, then the copy or move will fail. If you want to override an existing target path, you can use `REPLACE_EXISTING` option. If you want to copy all the file properties, you can use `COPY_ATTRIBUTES` option. You can also select both attributes as shown below:
 - `Files.copy(fromPath, toPath, standardCopyOption.REPLACE_EXISTING, standardCopyOption.COPY_ATTRIBUTES);`



Copy, move, and delete files

- ❑ The move operation can be defined as atomic, so that either the move operation completes successfully or the source file continues to remain in its original position. Specifically, you can use `ATOMIC_MOVE` options:
 - `Files.move(fromPath, toPath, standardCopyOption.ATOMIC_MOVE);`
- ❑ Finally, deleting a file can be called:
 - `Files.delete(path);`
- ❑ If the file to be deleted does not exist, this method throws an exception. Therefore, you can use the following methods instead:
 - `boolean deleted = Files.deleteIfExists(path);`
 - This deletion method can also be used to remove empty directories.



Get document information

▣ The following static methods will all return a boolean value that represents the result of checking a certain property of the path:

- `exists()`
- `isHidden()`
- `isReadable()`, `isWritable()`, `isExecutable()`
- `isRegularFile()`, `isDirectory()`, `isSymbolicLink()`
- `Size()`



Chapter 8: Stream Input and Output

- Stream
- Input and output streams
- Java stream inheritance framework
- File operation
- Object input/output streams and serialization
- Data access object pattern



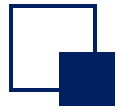
Object input and output and serialization

- ❑ When you need to store the same type of data, using a fixed-length record format is a good option. However, objects created in object-oriented programs rarely all have the same type. For example, you may have an array of Person records, but it actually contains instances of subclasses such as Student.
- ❑ We could certainly design our own data format to store this polymorphic set, but fortunately, we didn't have to.



Object input and output and serialization

- ❑ The Java language supports a general mechanism called object serialization, which writes any object out of the output stream and reads it back afterwards.
- ❑ Serialization vs. deserialization
 - The process of converting an object into a sequence of bytes is called serialize an object.
 - The process of restoring a byte sequence to an object is called deserialize an object.



Object serialization purpose

- ❑ Save an object's byte sequence permanently on a hard drive, usually in a file.
 - In many applications, certain objects need to be serialized so that they leave memory space and go to the physical hard drive for long-term storage. For example, the most common is the Session object in the web server, when there are 100,000 users accessing at the same time, there may be 100,000 Session objects, and the memory may be unbearable, so the web container will serialize some sessions to the hard disk first, and then restore the objects saved in the hard disk to memory.
- ❑ The byte sequence of an object is transmitted over the network.
 - When two processes are communicating remotely, they can send various types of data to each other.
 - Whatever type of data, it is transmitted over the network in the form of a binary sequence.
 - The sender needs to convert this Java object to a byte sequence in order to transmit it over the network; The receiver needs to restore the byte sequence to a Java object.



Serialization API in the JDK class library

- ❑ `Java.io.ObjectOutputStream` represents the object output stream, and its `writeObject` (`Object obj`) method can serialize the object specified by the parameter and write the resulting byte sequence to a target output stream.
- ❑ `ObjectOutputStream(OutputStream out)`
 - Creating an `ObjectOutputStream` allows you to write objects out to a specified `OutputStream`.
- ❑ `Void writeObject(Object obj)`
 - Writing the specified object to the `ObjectOutputStream` will store the specified object's class, the class's signature, and the values of all non-static and non-transient domains in the class and its superclasses.
- ❑ The object serialization steps are as follows:
 - Create an object output stream that wraps a different type of target output stream, such as a file output stream;
 - Write an object through the `writeObject()` method of the object output stream;



Serialization API in the JDK class library

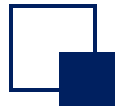
- ❑ `Java.io.ObjectInputStream` represents an object input stream, and its `readObject()` method reads byte sequences from a source input stream, deserializes them into an object, and returns them.
- ❑ `ObjectInputStream(InputStream in)`
 - Create an `ObjectInputStream` to read back object information from the specified `InputStream`.
- ❑ `Object readObject()`
 - Read an object from the `ObjectInputStream`. In particular, this method reads back the value of the object's class, the class's signature, and all non-static and non-instantaneous domains in this class and its superclass. The deserialization it specifies allows multiple object references to be restored.
- ❑ The object deserialization steps are as follows:
 - Create an object input stream that wraps a different type of source input stream, such as a file input stream;
 - Read the object via the `readObject()` method of the object input stream;



Serialization Cases

- ❑ The following Person class is defined, which implements the Serializable interface

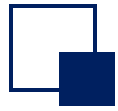
```
public class Person implements Serializable {  
    private String name;  
    private int age;  
    public Person(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
    //To make it easier to see objects, override the default toString()  
    @Override  
    public String toString(){  
        return "Person{" + "name=" + name + "\" +  ", age=" + age + "}";  
    }  
}
```



Serialization Cases

- ❑ The writeObject output of the ObjectOutputStream object is called to serialize the object

```
public class SeriDemo {  
    public static void main(String[] args) {  
        // ObjectOutputStream  
        try {  
            Person p1 = new Person("zhangsan", 30);  
            System.out.println(p1);  
            ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream("person.dat"));  
            oos.writeObject(p1);  
        } catch (Exception e) {  
            e.printStackTrace();  
        }  
    }  
}
```



Serialization Cases

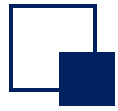
- ❑ Call the `readObject()` of the `ObjectInputStream` object to get the serialized object

```
//Create an ObjectInputStream input stream
try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream("person.dat"))){
    System.out.println("readObject(): ");
    Person zhangsan = (Person) ois.readObject();
    System.out.println(zhangsan);
} catch (Exception e) {
    e.printStackTrace();
}
```



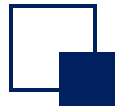
Chapter 8: Stream Input and Output

- Stream
- Input and output streams
- Java stream inheritance framework
- File operation
- Object input/output streams and serialization
- Data access object pattern



Data access object patterns

- ❑ The Data Access Object Pattern or DAO pattern is used to separate low-level data access operations from high-level business services.
- ❑ Participants in the Data Access Object pattern:
 - Data Access Object Interface: This interface defines the standard actions to be performed on a model object.
 - Data Access Object Entity Class: This class implements the above interface and is responsible for obtaining data from a data source, which can be a database, XML, or other storage mechanisms.
 - Model Object/Numeric Object: This object is a simple plain object that contains the get/set method to store the data retrieved through the use of DAO classes.



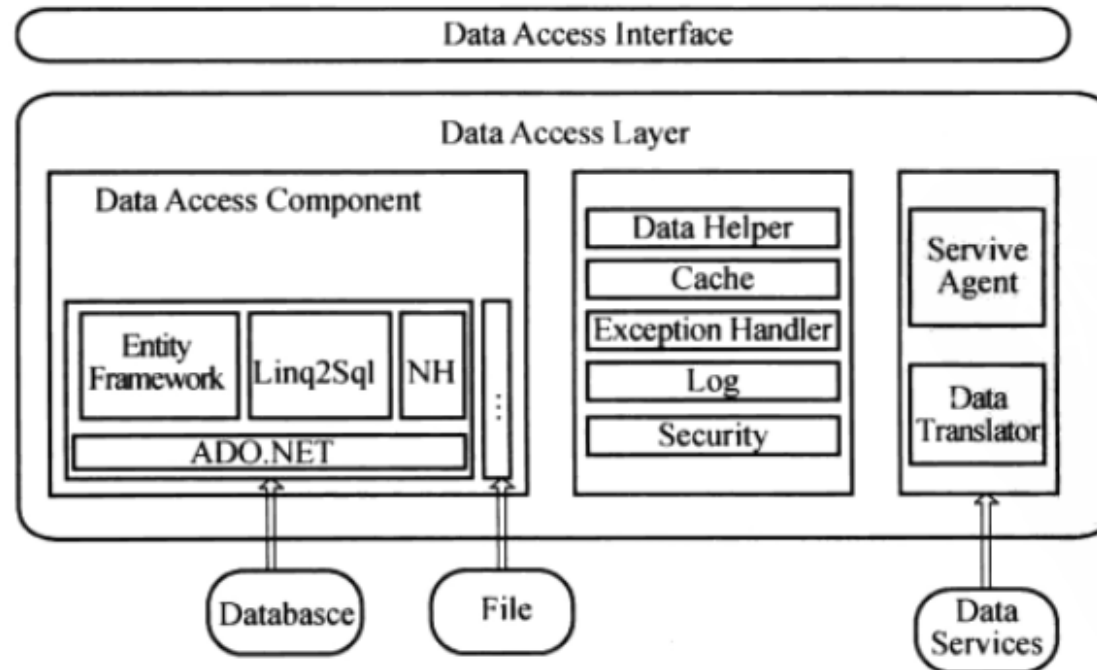
Data access object patterns

□ Advantages of the data access object pattern:

- Isolated Data Layer: Due to the addition of the DAO layer, it will not affect the interaction between services or entity objects and databases, and errors will be thrown exceptionally at this layer.

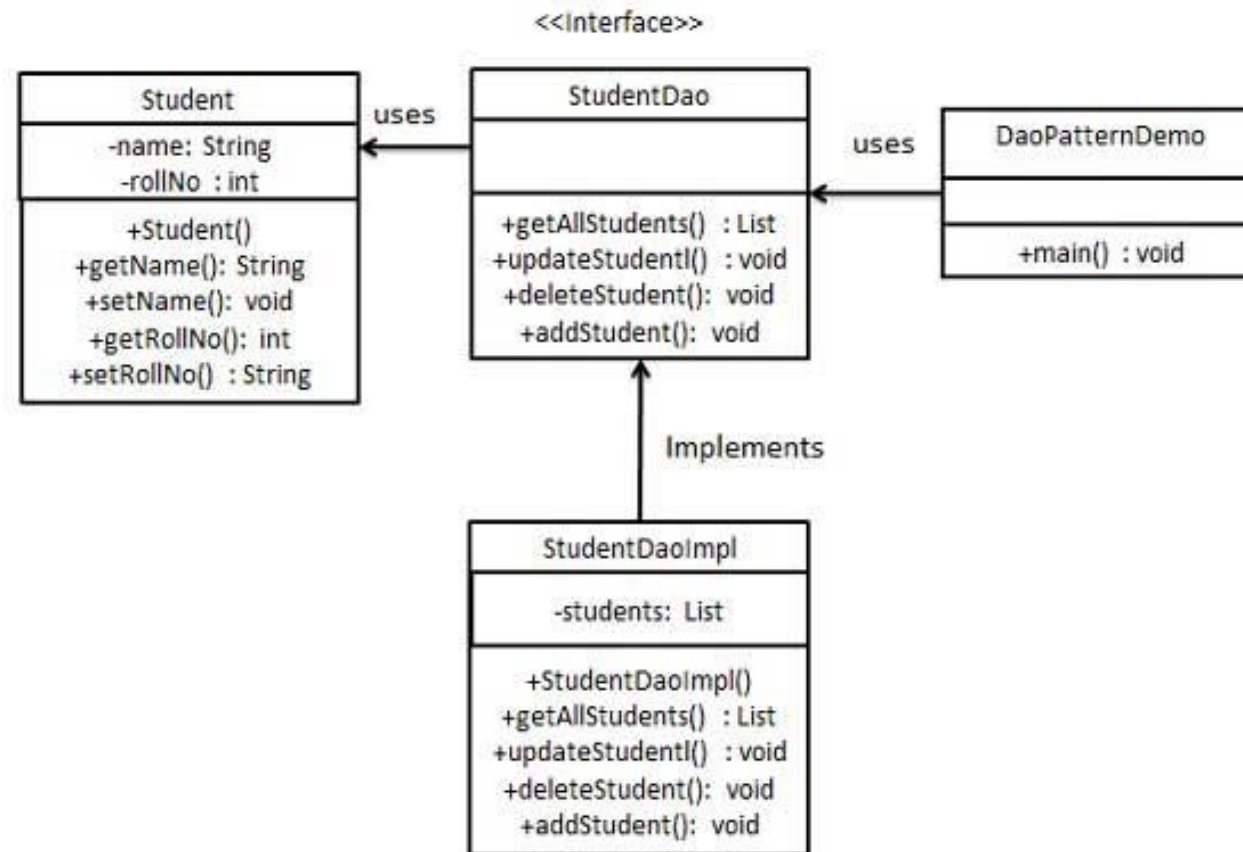
□ Disadvantage :

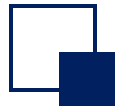
- Increased code volume: Adding one layer increases the amount of code (although this disadvantage is actually negligible)



Data access object pattern implementation

- ❑ We'll create a Student object as a model object or a numeric object. StudentDao is a data access object interface. StudentDaoImpl is an entity class that implements the interface for data access objects.
- ❑ DaoPatternDemo, our demo class uses StudentDao to demonstrate the use of data access object patterns.





Data access object pattern implementation

Step 1: Create a numeric object

```
1. public class Student {  
2.     private String name;  
3.     private int rollNo;  
4.  
5.     Student(String name, int rollNo){  
6.         this.name = name;  
7.         this.rollNo = rollNo;  
8.     }  
9.  
10.    public String getName() {  
11.        return name;  
12.    }
```

```
13.    public void setName(String name) {  
14.        this.name = name;  
15.    }  
16.  
17.    public int getRollNo() {  
18.        return rollNo;  
19.    }  
20.  
21.    public void setRollNo(int rollNo) {  
22.        this.rollNo = rollNo;  
23.    }  
24. }
```



Data access object pattern implementation

□ Step 2: Create a data access object interface

```
1. public interface StudentDao {  
2.  
3.     List<Student> getAllStudents();  
4.  
5.     Student getStudent(int rollNo);  
6.  
7.     void updateStudent(Student student);  
8.  
9.     void deleteStudent(Student student);  
10. }
```



Data access object pattern implementation

❑ Step 3: Create an entity class that implements the above interface

```
1. public class StudentDaoImpl implements StudentDao {
2.     //The list is used as a database
3.     List<Student> students;
4.
5.     public StudentDaoImpl() {
6.         students = new ArrayList<Student>();
7.         Student student1 = new Student("Robert", 0);
8.         Student student2 = new Student("John", 1);
9.         students.add(student1);
10.        students.add(student2);
11.    }
12.    @Override
13.    public void deleteStudent(Student student) {
14.        students.remove(student.getRollNo());
15.        System.out.println("Student: Roll No " + student.getRollNo() + ",
deleted from database");
16.    }
```



Data access object pattern implementation

❑ Step 3: Create an entity class that implements the above interface

```
17.//Retrieve the list of students from the database
18.    @Override
19.    public List<Student> getAllStudents() {
20.        return students;
21.    }
22.
23.    @Override
24.    public Student getStudent(int rollNo) {
25.        return students.get(rollNo);
26.    }
27.
28.    @Override
29.    public void updateStudent(Student student) {
30.        students.get(student.getRollNo()).setName(student.getName());
31.        System.out.println("Student: Roll No " + student.getRollNo()
32.            + ", updated in the database");
33.    }
34.}
```



Data access object pattern implementation

❑ Step 4: Create a StudentDao to demonstrate the use of the data access object pattern

```
1. public class DataAccessObjectPatternDemo {  
2.     public static void main(String[] args) {  
3.         StudentDao studentDao = new StudentDaoImpl();  
4.         //Output all students  
5.         for (Student student : studentDao.getAllStudents()) {  
6.             System.out.println("Student: [RollNo : "  
7.                 + student.getRollNo() + ", Name : " + student.getName() + " ]");  
8.         }  
9.         System.out.println();  
10.        //Update students  
11.        Student student = studentDao.getAllStudents().get(0);  
12.        student.setName("Michael");  
13.        studentDao.updateStudent(student);  
14.        System.out.println();  
15.        //Get students  
16.        studentDao.getStudent(0);  
17.        System.out.println("Student: [RollNo : " + student.getRollNo() + ",  
                                Name : " + student.getName() + " ]");  
18.    }  
19. }
```



Data access object pattern implementation

Step 5: Verify the output

```
1.Student: [RollNo : 0, Name : Robert ]  
2.Student: [RollNo : 1, Name : John ]  
3.Student: Roll No 0, updated in the database  
4.Student: [RollNo : 0, Name : Michael ]
```

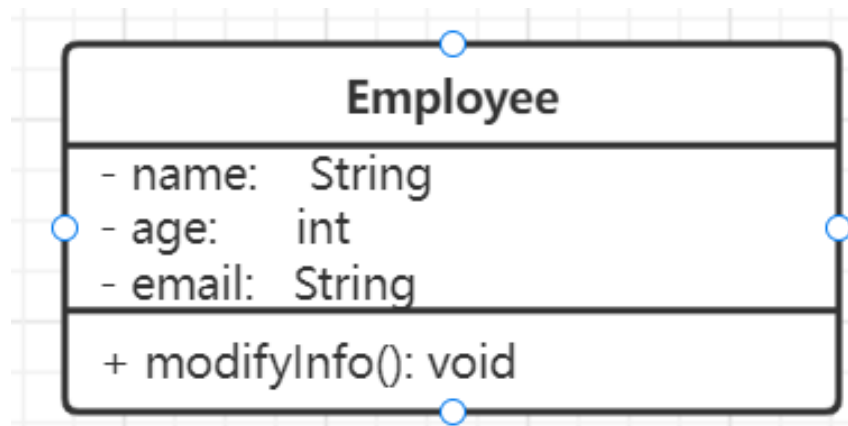


Chapter 8: Stream Input and Output

- Stream
- Input and output streams
- Java stream inheritance framework
- File operation
- Object input/output streams and serialization
- Data access object pattern

UML class diagram

Class Diagram: A static model used to show classes, interfaces, and the static structure and relationships between them in a system.



```
public class Employee {  
  
    private String name;  
    private int age;  
    private String email;  
  
    public void modifyInfo() {  
        .....  
    }  
}
```

UML class diagram

Relationship between classes: Association, generalization, implementation, and dependencies. Among them, associations are divided into general associations, aggregation relationships and combination relationships.

01 ↑

generalize

implementation, Subclasses inherit all behaviors and attributes of the parent class.

02 ↑

implement

The relationship between a class and an interface

03 ↓

depend

One usage relationship, the implementation of a class requires the assistance of other classes.

04 ↓

associate

The connection between one class of objects and another.

05 ↓

aggregate

The relationship between the whole and the part, and the part can exist separately from the whole.

06 ↓

combine

The relationship between the whole and the part, but the part cannot exist alone without the whole.

UML class diagram

01 ↑ generalize

Inheritance, subclasses inherit all behaviors and attributes of the parent class. Such as: tigers and animals.

UML class diagram



The relationship between a class and an interface indicates that the class is the implementer of all the characteristics and behaviors of the interface. For example: birds and flight.

UML class diagram

03



depend

Usage: the implementation of a class requires the assistance of other classes. For example: driver and car.

UML class diagram

04



associate

A referential relationship between objects that represents the connection between one class of objects and another. For example: teachers and students.

UML class diagram



The relationship between the whole and the part, and the part can exist separately from the whole. For example: cars and tires.

UML class diagram



The relationship between the whole and the part, and the part can exist separately from the whole. For example: cars and tires.

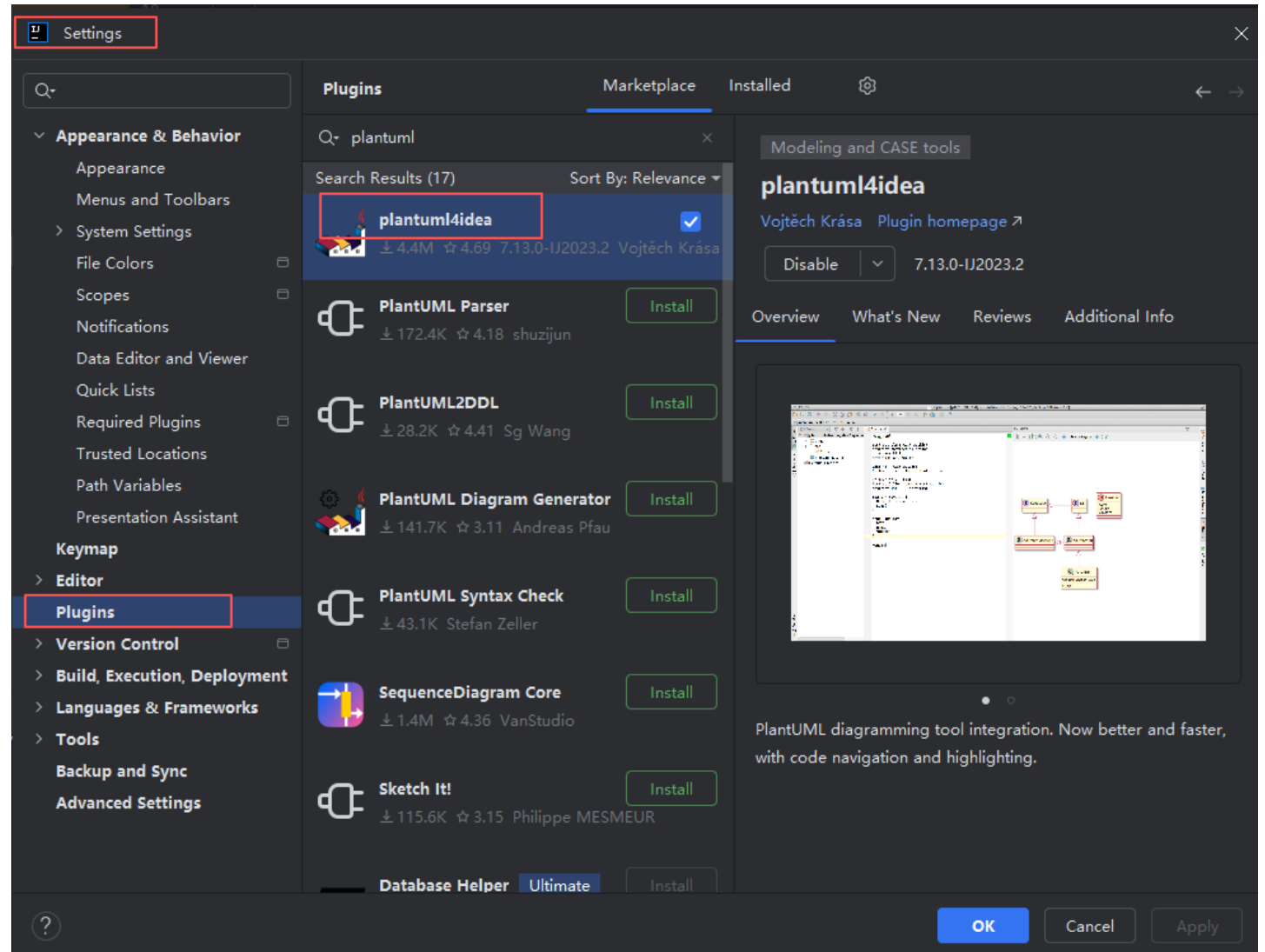
UML class diagram



The relationship between the whole and the part, but the part cannot exist alone without the whole. For example: companies and departments.

PlantUML plugin

- PlantUML is an open-source project that supports the rapid drawing of UML structure diagrams and behavior diagrams such as class diagrams, time series diagrams, and use case diagrams.

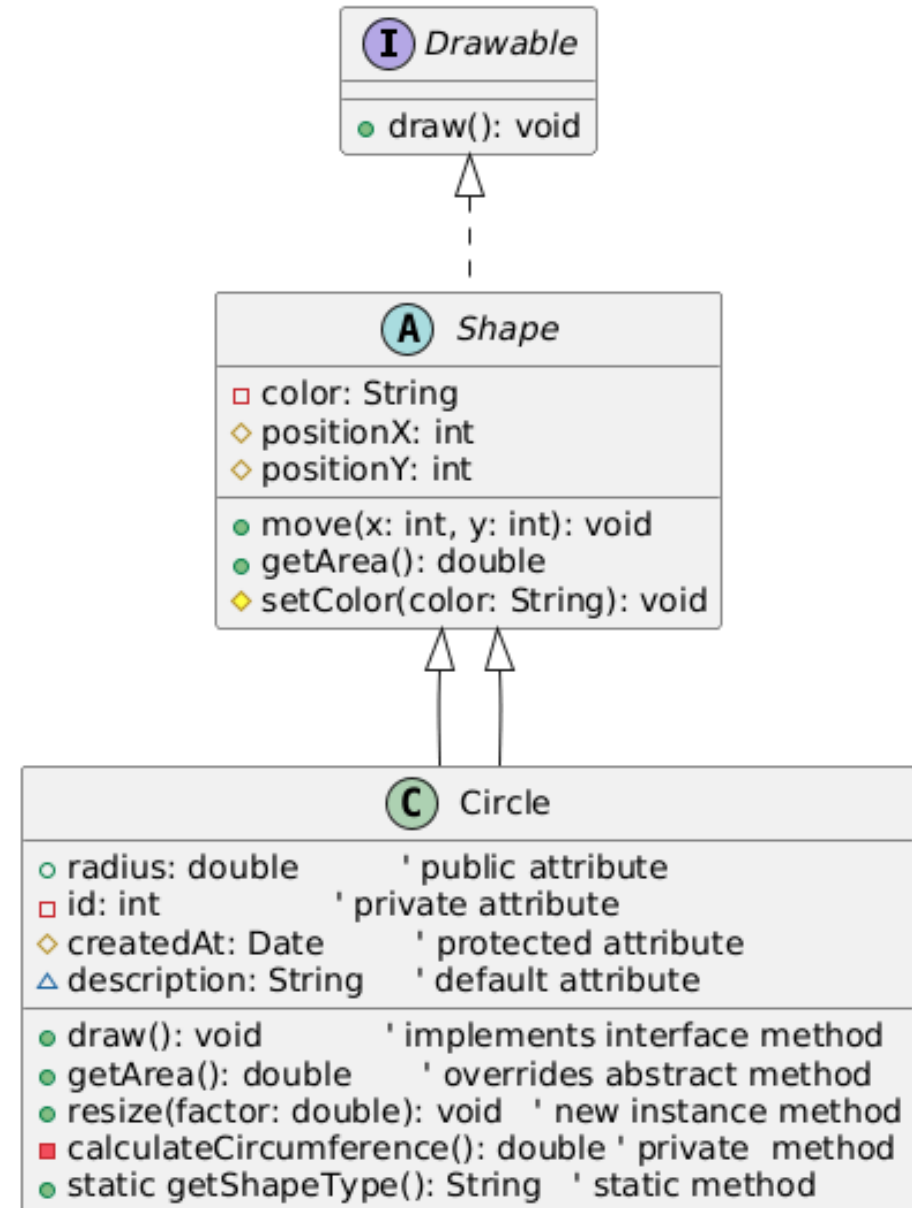


PlantUML plugin

- For PlantUML class diagrams and related syntax, please refer to the official website:

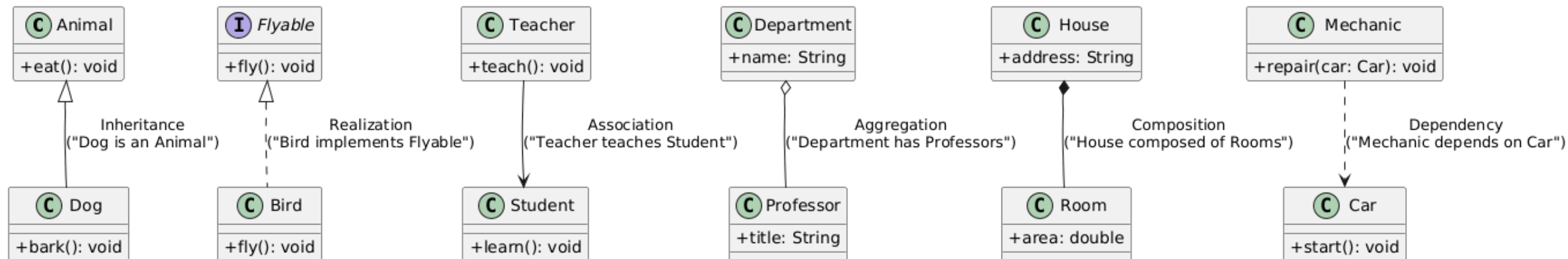
<https://plantuml.com/class-diagram>

- Interfaces, abstract classes, and class drawing methods:



PlantUML plugin

➤ How to draw 6 relationships between classes:



Question

- Design a UML class diagram for a simple Student–Course System with the following requirements:
 - A Teacher teaches Courses.
 - A Student can take Courses.
 - Both Student and Teacher share common information such as name and id.

Question

